主机发起一切. USB 是严格的主从结构，设备永远不能主动说话。即使是名为“interrupt”的传输类型，也只是“主机承诺每隔 `bInterval` 毫秒来问一次”：主机发 IN 令牌，设备有数据就回 DATA，没有就回 NAK。所谓“键盘中断”，其实是主机控制器替 CPU 轮询，问到数据后才用 IRQ 9 通知 CPU。

host channel 与 DMA. DWC2 有 8 个 host channel，每个 channel 是一组寄存器，描述“对哪个设备的哪个端点做一次什么传输”：`HCCHAR`（设备地址、端点号、方向、传输类型、最大包长、是否低速）、`HCSPLT`（split 事务参数）、`HCTSIZ`（字节数、包数、DATA0/1）、`HCDMA`（缓冲区的总线地址）、`HCINT`（完成、ACK、NAK、NYET、错误等状态）。本项目固定分工：ch0 做枚举时的 control 传输，ch6 给键盘的 interrupt-IN，ch4 给 MT7601U 的 bulk-IN。和 Arasan 不同，DWC2 自己用 DMA 读写内存，所以缓冲区在提交前要 clean、完成后要 invalidate 缓存，地址也要换成 `0xC0000000|PA` 的总线地址。

一切都在 Hub 后面. 树莓派 3 的 SoC 只有一个 USB 口，直接接板载 LAN9514，它内部是一个高速 Hub 加一块 USB 以太网卡（在 3B+ 上还有第二层 Hub）。外面插的键盘一般是低速（1.5 Mb/s）设备，而 DWC2 和 Hub 之间跑的是高速（480 Mb/s）。两种速率之间靠 Hub 里的事务转换器（TT）衔接，这就是后面 split transaction 的由来。

7.4.2 内核软件框架：五层栈与三条抽象边界

上一小节把驱动粗分成 HCD、USB core、类驱动三层；落到代码里，一份键盘输入实际要穿过**五层**软件栈，外加一层正交的通用设备模型（图 7.4）。这套分层照搬 Linux 的 `drivers/usb` 加 `drivers/base`：相邻两层之间由一张函数指针表（ops）隔开，上层只认这张表、不认下层的具体实现——于是换一块控制器（改 `dwc2.c`）不动任何类驱动，加一个类驱动（`usbmouse.c`、`mt7601u.c`）也不动 core 和 HCD。

边界 1：`usb_host_ops`——core 与 HCD 之间. 这是本项目里 Linux `usb_hcd` 的等价物（`usb.h`）。core 层只通过这张表驱使硬件：下行有 `submit_urb/kill_urb`（异步 URB）和 `control/bulk/bulk_rx_arm`（同步请求）；上行则只留一个入口 `usb_hcd_giveback_urb()`——HCD 完成一笔传输后必须回到这个函数，由 core 再把结果转交给发起者的 `urb->complete` 回调。整张表在 `dwc2.c` 里实例化为 `dwc2_usb_ops`，枚举出的每个设备、乃至合成的根 Hub，都把 `udev->ops` 指向它。

边界 2：`urb` 与 `urb->complete`——core 与类驱动之间. 类驱动不直接碰寄存器，而是填一个 Linux 风格的 `struct urb`（端点号、方向、缓冲区、`complete` 回调），交给 `usb_submit_urb()`。下行链是：

```
usbkbd_submit / usb_submit_urb(urb)          # usbkbd.c -> usb.c
  -> urb->dev->ops->submit_urb(urb)          # 函数指针跳转：core -> HCD
    -> dwc2_submit_urb                       # dwc2.c
      -> 占用 interrupt_rx[] 槽位，编程 ch6 的 HCCHAR/HCSPLT/HCTSIZ/HCDMA
```

上行链把完成一路送回用户空间，中间两次跨越“硬中断 → 工作队列”的上下半部边界：

```
IRQ 9 -> dwc2_irq -> dwc2_interrupt_urb_work # 硬中断只 schedule_work
  -> usb_hcd_giveback_urb(urb, status, len)  # HCD -> core（唯一入口）
    -> urb->complete(urb)                    # 函数指针跳转：core -> 类驱动
      -> usbkbd_irq_complete -> queue_work(usbkbd_wq, rx_work)
        -> usbkbd_rx_work -> input_report_key()/input_sync()   # 边界3
          -> evdev_pass_event -> wake_up(client)
            -> read(/dev/input/eventN) 返回一个 input_event    # 用户空间
```